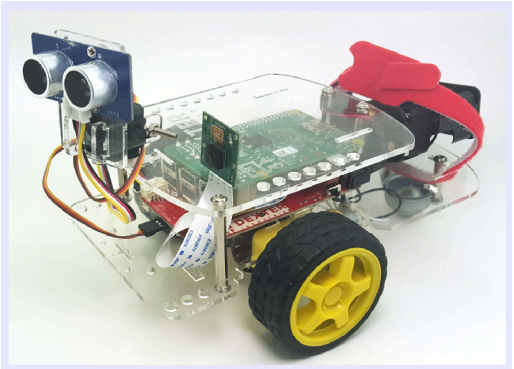
Rapport Projet de Développement R2D2

Introduction :
L'objectif de ce projet est de manipuler le robot Dexter : le faire rouler, tourner… en utilisant Python.
Le projet a pour but principal de pouvoir élaborer des lignes de codes qui permettent d'effectuer ces différentes actions :
- Suivre une balise
- S'approcher le plus vite possible d'un obstacle
- Tracer un carré

Le robot est constitué de 2 roues et 2 moteurs, un capteur de distance (vers l'obstacle le plus proche), un accéléromètre pour modifier sa vitesse, d'une caméra, d'un Raspberry Pi qui est un petit ordinateur qui permettra de lui donner des instructions Python etc…



Pour cela, nous avons tout d'abord créé un environnement de simulation sur Python qui permet de visualiser les déplacements du robot grâce à une interface graphique.
On a dû ensuite ajouter des "traducteurs" pour associer les codes de la simulation avec le robot réel.

Mode d'emploi : (Interface graphique)
Après avoir téléchargé le repo Github "Projet-UL2IN013", on aura accès à 3 répertoires :
- "Compte Rendu" qui contient un rapport des séances chaque semaine.
- "OldCode" qui contient d'anciens codes qui ont été abandonnés.
- "CurrentCode" qui contient les codes fonctionnels.

Une fois rendu sur le répertoire "CurrentCode", pour pouvoir lancer la simulation sur l'interface graphique, on pourra éxecuter le fichier "main.py" en tapant :
"python3 main.py"
ce qui ouvrira une fenêtre où un robot (représenté par un rectangle) se déplacera.
Pour pouvoir manipuler le robot, c'est-à-dire, le faire avancer, tourner, faire un carré…, il suffit d'accéder au répertoire "R2D2" dans lequel se trouve le répertoire "controller" qui contient le fichier "controller.py" et d'insérer à la ligne 12 les différentes actions que proposent le robot (à l'intérieur des crochets "[ ]" et séparées par une virgule) :
- Avancer(*distance*, *vitesse*, self.trad); distance : la distance que le robot parcourt (en m)
vitesse : la vitesse du robot (en m/s)

- Tourner(*angle*, self.trad); angle : l'angle par laquelle le robot tourne

- Carre(*cote*, *vitesse*, self.trad); cote : la "taille" d'un côté du carré
vitesse : la vitesse du robot

-Approcher_mur(self.trad) qui avance jusqu'à un obstacle.

Mode d'emploi : (Robot)
Pour pouvoir manipuler le robot réel, il faut tout d'abord se connecter au robot (cf. README du projet).
Une fois connecté, pour pouvoir mouvoir le robot, la méthode est identique à celle présentée précédemment. (modifier "controller.py").
Il suffit ensuite de lancer le fichier "mainreel.py" en tapant :
"python3 mainreel.py"

Equations utilisées :
Vitesse = distance / temps

Distance parcourue (par la roue) = angle relatif parcouru (par la roue) / 360 * π * diamètre de la roue.

Vitesse angulaire = (vitesse roue gauche - vitesse roue droite) / écart des 2 roues

Points à améliorer :
- Nous aurions pu faire en sorte que le robot puisse faire 2 actions simultanément, par exemple qu'il puisse tourner et avancer en même temps (comme une voiture ordinaire). Le nôtre peut seulement exécuter des instructions séquentiellement.

- Nous aurions pu ajouter la gestion de collision dans le robot réel. En effet, ce problème a déjà été implémenté dans la simulation mais dans l'environnement réel il aurait fallu utiliser le capteur de distance et la caméra qui fonctionnaient mal.

-Nous aurions pu ajouter plus de tâches à faire effectuer au robot comme une chasse au trésor ou le suivi d'une balise.

Schéma fonctionnel :

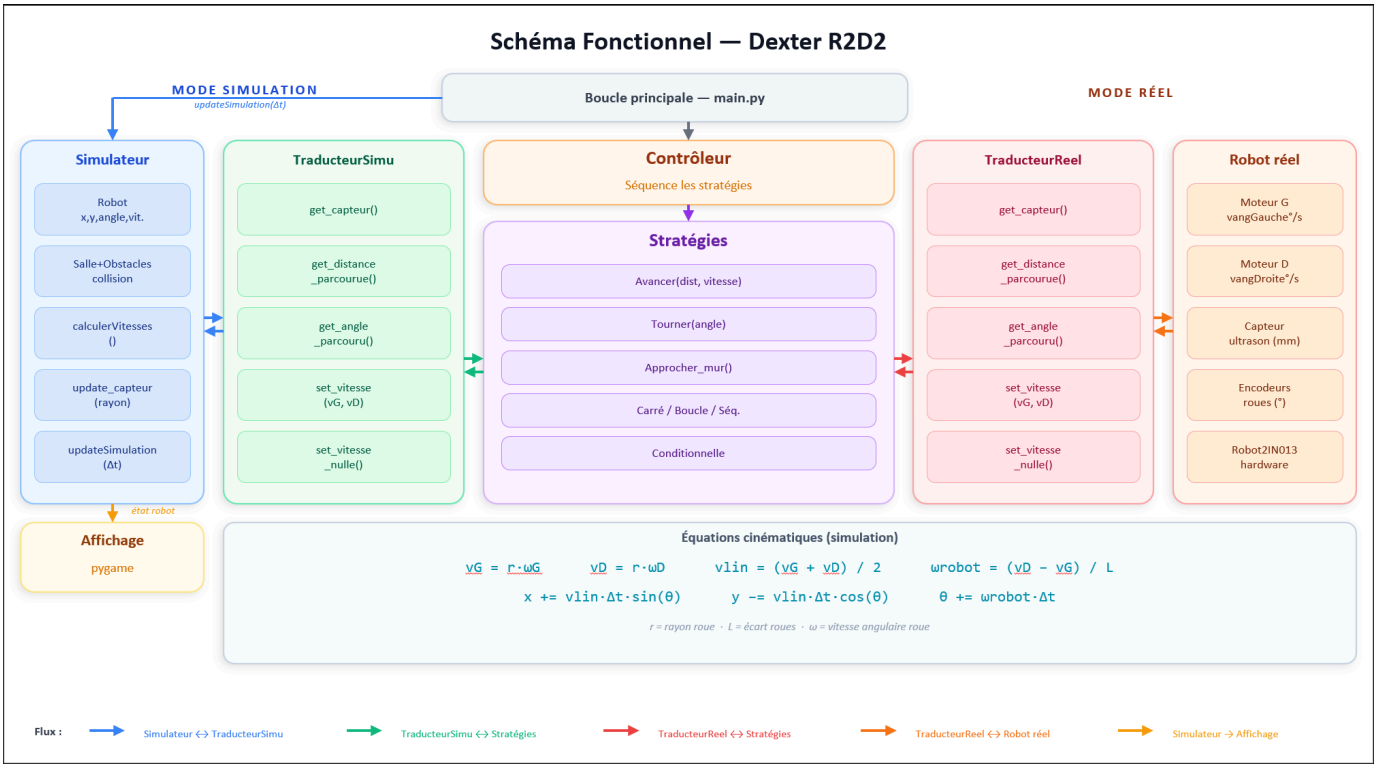
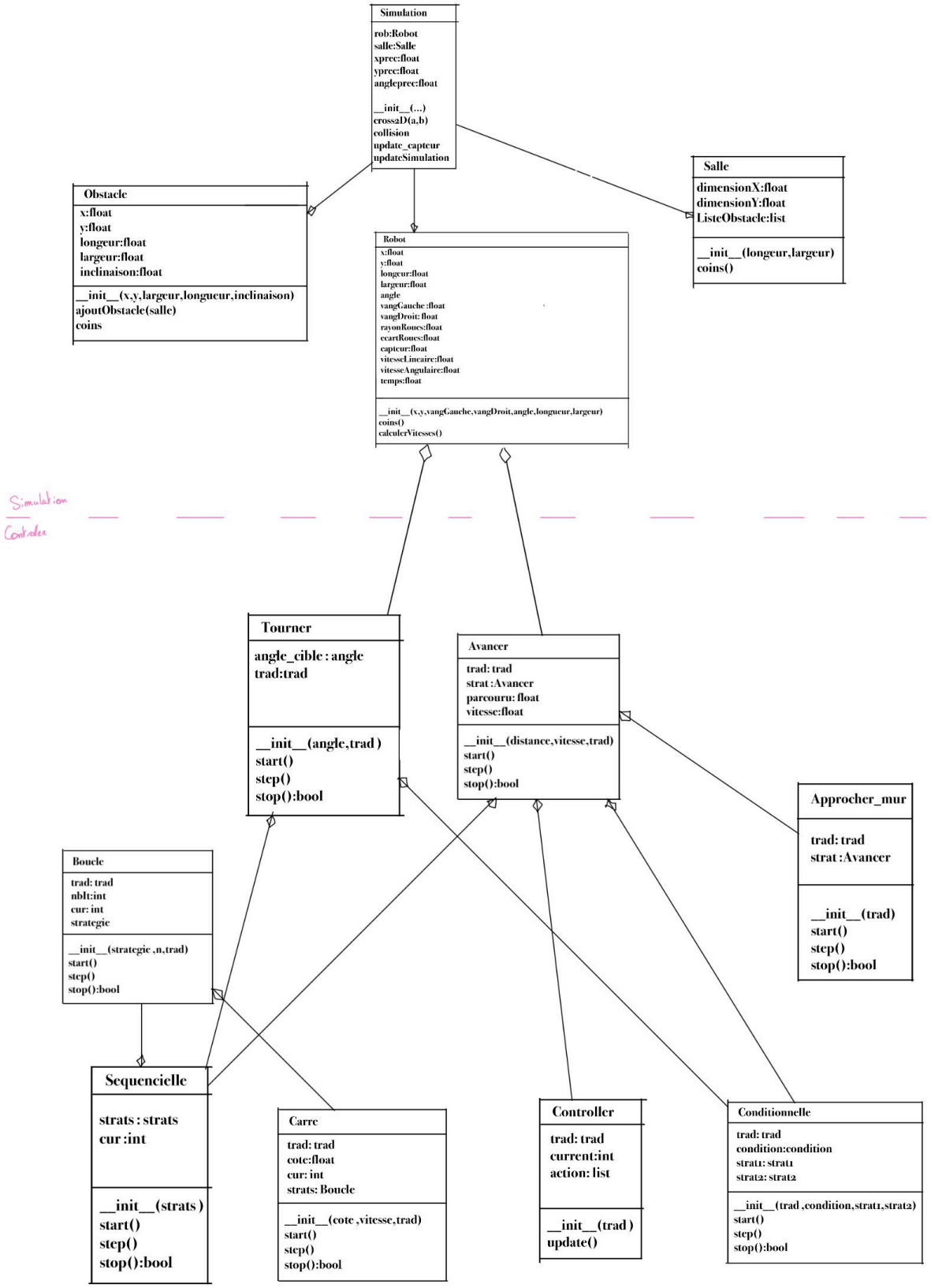


Schéma d'architecture (ULM) :



DAI Clément
ZHOU Mattéo
WANG Martial
COIPEL Thomas
AKKOYUN Beyza